

A Faster Directed Single-Source Shortest Path Algorithm

Ran Duan *

Xiao Mao †

Xinkai Shu ‡

Longhui Yin *

February 2026

Abstract

This paper presents a new deterministic algorithm for single-source shortest paths (SSSP) on real non-negative edge-weighted directed graphs, with running time $O(m\sqrt{\log n} + \sqrt{mn \log n \log \log n})$, which is $O(m\sqrt{\log n \log \log n})$ for sparse graphs. This improves the recent breakthrough result of $O(m \log^{2/3} n)$ time for directed SSSP algorithm [Duan, Mao, Mao, Shu, Yin 2025].

1 Introduction

The single-source shortest path (SSSP) problem is one of the most foundational problems in graph theory, whose algorithms have significant improvements since the 1950s. Given a graph $G = (V, E)$ with n vertices and m non-negative real-weighted edges, the goal is to compute the distance from a source vertex s to every vertex $v \in V$. We work under the *comparison-addition* model, where only comparison and addition on edge weights are allowed – a natural assumption that align with real-weighted inputs.

The textbook algorithm by Dijkstra [Dij59], when implemented with a Fibonacci heap [FT87], solves the problem in $O(m + n \log n)$ time. In addition to computing the distances, Dijkstra’s algorithm also produces the ordering of vertices. Haeupler, Hladík, Rozhoň, Tarjan and Tětek [HHR⁺24] demonstrated that Dijkstra’s algorithm is universally optimal if this order is required. On the other hand, if only the distances are needed, there were no substantial improvements until recently. Duan, Mao, Shu and Yin [DMSY23] proposed a randomized algorithm with $O(m\sqrt{\log n \log \log n})$ running time for undirected graphs, which was subsequently derandomized by Yan [Yan26]. Later, a groundbreaking result by Duan, Mao, Mao, Shu and Yin [DMM⁺25] introduced a deterministic $O(m \log^{2/3} n)$ -time algorithm for directed graphs, first to break the $O(n \log n)$ time bound in sparse graphs.

In this work, we further improve the running time of SSSP to $O(m\sqrt{\log n} + \sqrt{mn \log n \log \log n})$ on directed graphs, matching the previous results for undirected graphs.

Theorem 1.1. *Single-source shortest path on directed graphs with real non-negative edge weights can be solved in deterministic $O(m\sqrt{\log n} + \sqrt{mn \log n \log \log n})$ time.*

An equivalent statement is, $O(m\sqrt{\log n})$ time for $m \geq n \log \log n$, $O(\sqrt{mn \log n \log \log n})$ for $m < n \log \log n$, and $O(n\sqrt{\log n \log \log n})$ for $m = O(n)$.

*Tsinghua University. Email: duanran@mail.tsinghua.edu.cn, ylh21@mails.tsinghua.edu.cn.

†Stanford University. Email: matthew99a@gmail.com.

‡Max Planck Institute for Informatics. Email: xshu@mpi-inf.mpg.de.

1.1 Technical Overview

Let us begin with Dijkstra’s algorithm, which maintains a “frontier” S of vertices, such that if a vertex is “incomplete” — meaning its current distance label is greater than its true distance — then its shortest path visits some complete vertex in S . In other words, the frontier encloses a set of vertices sufficient for finding the shortest paths of all remaining incomplete vertices.

Dijkstra’s algorithm each time extracts from the frontier the vertex with smallest distance label, which is guaranteed to be complete. It then relaxes all out-going edges and adds these neighbors to frontier. This produces an ordering of all vertices by distances, creating an inherent $\Omega(n \log n)$ sorting barrier.

The breakthrough in [DMM⁺25] overcomes this barrier by introducing a divide-and-conquer procedure with $(\log n)/t$ levels, namely Bounded Multi-Source Shortest Path (BMSSP), which operates on a “frontier” S with respect to a bound B . BMSSP aims to settle all vertices in $\tilde{U} = \tilde{U}(B, S) = \{v : \text{dis}(v) < B \text{ and its shortest path visits some vertex in } S\}$. It recursively invokes subcalls of BMSSP on sets consisting of the smallest $1/2^t$ fraction of vertices extracted from the frontier. A naïve implementation would still spend $\Theta(t)$ time on each frontier vertex, yielding a total running time of $\Theta(\log n)$ per vertex.

To solve this problem, [DMM⁺25] reduces the size of frontier S by a factor $1/k$ regarding to the target set $\tilde{U}$, by running Bellman-Ford algorithm for k iterations. Any remaining incomplete vertex must lie in the shortest path tree of some $x \in S$ with size at least k . Such vertices x are called “pivots”, and their number is bounded by $|\tilde{U}|/k$. However, this approach still poses a bottleneck since Bellman-Ford requires $\Omega(k)$ time per vertex on average. This cost seems unavoidable, because the algorithm needs to determine, for each vertex $v \in \tilde{U}$ which pivot’s shortest path tree it belongs to.

A key observation is that, such explicit mapping is unnecessary. Our new insight is that we can construct a spanning forest that connects the frontier S via local Dijkstra searches from S , and partition it into edge-disjoint subtrees of sizes $\Theta(k)$. This idea is inspired by [DLWX18] that tackles the single-source bottleneck path (SSBP) problem, and reduces the cost of finding pivots from $O(k)$ to $O(\log k)$ per vertex. Then, for each subtree, the algorithm only tracks the vertex with the smallest distance label, referred to as a “pivot”, which serves as a handle for the vertices in its corresponding subtree. Then we are able to maintain $1/k$ of the vertices in S in the partial-sorting data structure, and meanwhile efficiently select a minimal subset of the whole frontier. For other vertices, we only need to check whether they fit the range of the subset without sorting them. Then the time for inserting a vertex in the data structure can be amortized to an edge on the subtree containing it, but that edge is not supposed to be in recursive subcalls. So the overall running time is roughly $O(m\sqrt{\log n})$. However, in this recursive procedure for shortest path, the upper bound B used for each invocation is dynamically generated. If the range by bound B contains too many vertices with respect to S , the algorithm can only perform partial execution, and some vertices may be reinserted into S after extraction. This behavior must be handled carefully to achieve the running time mentioned above.

1.2 Related Works

Single-source shortest path has been extensively studied in various special cases. Pettie and Ramachandran [PR05] proposed an algorithm running in $O(m\alpha(m, n) + \min\{n \log n, n \log \log r\})$ time for undirected graphs, where α is the inverse-Ackermann function and r is the max-min ratio of edge weights. In word-RAM model with bounded integer edge weights, a sequence of works [FW93, FW94, Tho00a, Ram96, Ram97, Tho00b, Hag00] culminated in Thorup’s linear-time algorithm for undirected graphs [Tho99] and $O(m + n \log \log \min\{n, C\})$ -time for directed graphs [Tho04], where C is the maximum edge weight.

If negative edge weights are allowed, the Bellman-Ford algorithm [Bel58] needs $O(mn)$ time. Beyond this, there are nearly linear time algorithms for integer weights (with respect to bit complexity) [CKL⁺22, BNWN22, BCF23], and strongly subcubic time algorithms for real weights [Fin24, HJQ25a, HJQ25b].

2 Preliminaries

The single-source shortest path problem is defined on a directed graph $G = (V, E)$ with a non-negative weight function $w : E \rightarrow \mathbb{R}_{\geq 0}$ (typically denoted w_{uv}) assigned to each edge. Let $n = |V|$ and $m = |E|$ denote the number of vertices and edges respectively. Given a source $s \in V$, the goal is to compute the length of the shortest path from s to every vertex $v \in V$. Without loss of generality we assume that every vertex is reachable from s , so $m \geq n - 1$.

For clarity, we restate two relevant definitions. A *directed tree* is a directed acyclic graph whose underlying undirected graph is a tree. An *arborescence* rooted at some vertex v is a directed tree such that every other vertex can be reached from v via exactly one directed path.

2.1 Degree reduction

For any degree bound $3 \leq \delta \leq \frac{m}{n}$, we preprocess G in $O(m)$ time to a graph with $O(m)$ edges, $O(m/\delta)$ vertices, and maximum (in- and out-) degree bounded by δ , while preserving all shortest path lengths. A classical way to achieve this, similar to that in [Fre83], proceeds as follows:

- Substitute each v with a zero-weighted cycle C_v of $\left\lceil \frac{\Delta_v}{\delta-2} \right\rceil$ vertices, where Δ_v denotes the degree of v . For each v 's neighbor u (either incoming or outgoing), assign one vertex in C_v to represent the edge between them, denoted as x_{vu} ; make sure each vertex in C_v represents at most $\delta - 2$ different x_{vu} .
- For every edge $(u, v) \in G$, add a directed edge from vertex x_{uv} to x_{vu} with weight w_{uv} .

Note that when $m \geq n \log n$, Dijkstra's algorithm implemented with Fibonacci heap already reaches the optimal time $O(m + n \log n) = O(m)$, thus it suffices to consider the regime $m \leq n \log n$ in our algorithm.

2.2 Comparison-Addition Model

Our algorithm works under the *comparison-addition* model, in which edge weights can only be compared or added, each taking $O(1)$ time. No other arithmetic or algebraic operations on edge weights are allowed.

2.3 Edge Relaxation and Tie-Breaking

We maintain global distance labels $d[v]$ for all vertices $v \in V$. Initially $d[s] = 0$ and $d[v] = \infty$ for all $v \neq s$. All updates to $d[v]$ are through relaxation: $d[v] \leftarrow \min\{d[v], d[u] + w_{uv}\}$ of some edge $(u, v) \in E$. A relaxation is *valid*, if $d[v]$ was updated, namely $d[u] + w_{uv} \leq d[v]$. In our algorithm, we also introduce an upper bound B and require $d[u] + w_{uv} < B$ to prevent immature modification. Hence, $d[v]$ is non-increasing and always corresponds to the length of some path from s to v . Let $\text{dis}(v)$ denote the length of the true shortest path from source s to v , then $d[v] \geq \text{dis}(v)$ always holds.

Similar to previous works [DI04, DMM⁺25], for clarity in presenting the algorithm, we define a tie-breaking rule for comparing distance labels in order to:

1. Keep the structure of the shortest path tree throughout the algorithm;
2. Establish a relative ordering among the vertices with identical $d[\cdot]$.

In principle, we treat a path of length l that traverses q edges, represented by the vertices $v_0 = s, v_1, \dots, v_q$, as a tuple $\langle l, q, v_q, v_{q-1}, \dots, v_0 \rangle$, where vertices are listed in reverse order, and sort them based on the lexicographical order. However, it turned out that the first 4 items of the above tuple are sufficient for tie-breaking: path length (`length`), number of edges (`nEdges`), v itself (`curr`), and v 's predecessor (`pred`). This is because our algorithm performs such comparisons in the following scenarios:

Relaxing an edge (u, v) : If $u \neq d[v].\text{pred}$, even if `length` and `nEdges` are tied, it suffices to compare u and $d[v].\text{pred}$; if $u = d[v].\text{pred}$, then $d[u]$ has been updated to currently “shortest” and $d[v]$ must get updated accordingly;

Comparing two different $d[u]$ and $d[v]$ for $u \neq v$: In this case `curr` acts as a tie-breaker.

Upper bound B : In the algorithm the upper bound B is also represented by such a 4-tuple.

Algorithm 1 illustrates how an edge (u, v) is relaxed. Note that an upper bound B is introduced to prevent premature update of vertices whose tentative distances exceed the current bound in the main algorithm.

Algorithm 1: Relaxation of an edge $(u, v) \in E$ with an upper bound B

```

1 Function Relax(vertex  $u$ , vertex  $v$ , upper bound  $B$ ):
    //  $B$  is introduced to prevent early update of vertices out of bound
2   if  $d[u] + w_{uv} \leq d[v]$  and  $d[u] + w_{uv} < B$  then
3      $d[v] \leftarrow d[u] + w_{uv}$ 
4     return True
5   return False
6 Tuple  $d$  :  $\langle \text{length}, \text{nEdges}, \text{curr}, \text{pred} \rangle$ 
7 Function Addition(distance label  $d[u]$ , edge weight  $w_{uv}$ ):
8   return  $\langle d[u].\text{length} + w_{uv}, d[u].\text{nEdges} + 1, v, u \rangle$ 
9 Function Comparison(distance label  $d[u]$ , distance label  $d[v]$ ):
10  return Lexicographically compare  $\langle \text{length}, \text{nEdges}, \text{curr}, \text{pred} \rangle$  of  $d[u]$  and  $d[v]$ 

```

2.4 Completeness and Frontier

For a vertex v , if $d[v] = \text{dis}(v)$, we say v is *complete*. If all vertices in a set are complete, we say that set is complete. We need to specify the time step when referring to $d[v]$ and completeness because they are sensitive to the algorithm progress.

Let $S \subseteq V$ be a set of vertices and B be a bound. We define $\tilde{U} = \tilde{U}(B, S)$ the set of every vertex v such that $\text{dis}(v) < B$ and the shortest path of v visits some vertex in S . The goal is to make $\tilde{U}$ complete.

Additionally, for a vertex set $U \subseteq V$ and any bound B , define $d_B[U] = \min(\{B\} \cup \{d[x] : x \in U\})$ and $\text{dis}_B(U) = \min(\{B\} \cup \{\text{dis}(x) : x \in U\})$. (B is introduced here in case U is an empty set.)

We introduce *frontier*, which is crucial for understanding our algorithm. This concept also appeared in [DMM⁺25]: at any moment and, for two sets $X, Y \subseteq \tilde{U}$, we say $\langle X, Y \rangle$ is a frontier for $\tilde{U}$, if any $v \in \tilde{U}$ satisfies at least one of the following:

1. $v \in X$ and v is complete;
2. the shortest path of v visits some *complete* vertex (possibly itself) in Y .

Note X is not necessarily complete; but any incomplete vertex of X must satisfy the second condition. Based on the definition, we can make the following observations:

Observation 2.1. At any given moment, if $\langle X, Y \rangle$ is a frontier for $\tilde{U} = \tilde{U}(B, S)$:

1. $\langle X, Y \rangle$ continues to be a frontier for $\tilde{U}$ ever since;
2. $d_B[Y] = \text{dis}_B(Y)$;

3. $\tilde{U}(B, Y) \subseteq \tilde{U}(B, S)$;
4. $\langle \emptyset, Y \rangle$ is a frontier for $\tilde{U}(B, Y)$;
5. $\tilde{U}(d_B[Y], S) \subseteq X$ and $\tilde{U}(d_B[Y], S)$ is complete;

Proof sketch: the first observation lies from the fact that when a vertex becomes complete, it continues to be complete ever since. The second observation is because the vertex with minimum distance label $d[\cdot]$ in Y must be complete. The third observation is because: for any vertex in $\tilde{U}(B, Y)$, its shortest path visits some vertex y in Y ; and the shortest path of y , as a prefix of the shortest path of v , owing to $Y \subseteq \tilde{U} = \tilde{U}(B, S)$, visits some vertex in S , so the shortest path of v visits S ; The fourth observation is because: for any incomplete vertex in $\tilde{U}(B, Y)$, it is also in $\tilde{U}(B, S)$, then its shortest path visits some complete vertex in Y ; for any complete vertex v in $\tilde{U}(B, Y)$, by definition its shortest path visits some vertex $y \in Y$, and y must be complete as v is complete. The fifth is because if the distance of a vertex is less than $d_B[Y]$, then it cannot visit any complete vertex in Y .

2.5 A Framework of Dijkstra-like Algorithms

We found that Dijkstra's algorithm, [DMM⁺25] and our algorithm can be restated under a unified framework:

- (1) Given a frontier $\langle X, Y \rangle$ for $\tilde{U} = \tilde{U}(B, S)$. Our goal is to make $\tilde{U}$ complete;
- (2) Identify a complete set $Z \subseteq \tilde{U}$; or make Z complete, possibly after performing some relaxations;
- (3) Relax the outgoing edges from Z and denote R as the set of out-going neighbors of Z but not in Z with distance less than B , and relaxations from Z to them are valid. Clearly $R \subseteq \tilde{U}$;
- (4) Update to a new frontier for $\tilde{U}$ as $\langle X', Y' \rangle \stackrel{\text{def}}{=} \langle X \cup Z, (Y \setminus Z) \cup R \rangle$;
- (5) Return to the first step and repeat the process until $Y = \emptyset$, then $\tilde{U} = X$ is complete.

We show that for Z given by (2), after operations in (3), $\langle X', Y' \rangle$ is indeed a frontier for $\tilde{U}$. Since adding vertices never violates frontier conditions, it suffices to verify for $v \in \tilde{U}$ whose shortest path visits some complete $u \in Z$ such that u is in Y but removed from Y' , v still satisfy one of the two frontier conditions.

- If $v \in Z$, we are already done;
- If $v \notin Z$, since $u \in Z$, along the shortest path from u to v , there must exist two adjacent vertices x and y such that $x \in Z$ and $y \notin Z$; then y is in R and added to Y' , ensuring that the shortest path of v still visits a complete vertex $y \in Y'$.

The efficiency of the algorithm depends critically on how we can efficiently identify Z or make Z complete. In Dijkstra's algorithm, the smallest vertex $\arg \min_{x \in Y} d[x]$ is guaranteed to be complete. Therefore, the algorithm organizes Y using a priority queue and simply selects such x to form Z . In [DMM⁺25] and our approach, the algorithm repeatedly selects a subset of smallest vertices S_i from the frontier, and recursively invoke a lower-level algorithm on S_i to obtain a complete set U_i , which is then used to update the frontier.

3 The Main Result

Recall that we work on a preprocessed graph of $O(m)$ edges, $O(m/\delta)$ vertices with max degree δ . Following the approach in [DMM⁺25], our algorithm is based on a divide-and-conquer framework in which, at each step, we attempt to extract the smallest $1/2^t$ fraction of vertices from the frontier ($t \leq \log n$).

We fix parameter t and define $k = \lceil t/\log t \rceil$, also ensuring $\delta \leq \log k = O(\log \log n)$ holds.¹ A more precise choice of t will be given later; roughly $t \approx \sqrt{\log n}$. We are now ready to present our main lemma.

Lemma 3.1 (Bounded Multi-Source Shortest Path). *Given an upper bound B , a vertex set S of size at most $t^2 2^{lt}$, and an integer $l \in [0, \lceil (\log n)/t \rceil]$. Suppose $\langle \emptyset, S \rangle$ is a frontier for $\tilde{U} = \tilde{U}(B, S)$.*

BMSSP(B, S, l) (Algorithm 3) outputs a new bound $B' (\leq B)$, a vertex set $U \subseteq \tilde{U}$ of size $O(t^3 2^{lt})$, a set of vertices $\mathcal{D}$ arranged by data structure as in Lemma 3.4 parameterized by $M := t2^{(l-1)t}$, in $O(|U|(l \log t + \delta t))$ time. After running Algorithm 3, $U = \tilde{U}(B', S)$ is complete and $\langle U, \mathcal{D} \rangle$ is a frontier for $\tilde{U}$. Moreover, one of the following is true:

- Full execution ($B' = B$): $\mathcal{D} = \emptyset$ and $U = \tilde{U}$;
- Partial execution ($B' < B$): $|U| = \Theta(t^3 2^{lt})$.

In the top layer, we call Algorithm 3 with parameters $S = \{s\}$, $B = \infty$ and $l = \lceil (\log n)/t \rceil$. Since $|U| \leq |V| = o(t^3 n)$, it is a full execution with all vertices complete in the end. Furthermore, the degree of each vertex in V is at most $O(\delta)$. Thus the total running time is $O(m(t + \frac{\log n \log t}{\delta t}))$.

With $t = \sqrt{\log n \log \log n / \delta}$ and $\delta = \frac{1}{4} \min\{\frac{m}{n}, \log \log n\}$, it reduces to $O(m\sqrt{\log n \log \log n / \delta})$. If $\frac{m}{n} \geq \log \log n$, it is $O(m\sqrt{\log n})$. If $\frac{m}{n} < \log \log n$, it is $O(\sqrt{mn \log n \log \log n})$.

3.1 Finding Pivots

Recall that we hope to manage the frontier $\langle \emptyset, S \rangle$ more efficiently. As mentioned in Section 1.1, Algorithm 2 tries to connect S with edge-disjoint $\Theta(k)$ sized sub-trees. We perform a local Dijkstra search from every vertex $x \in S$, only adding edges with valid relaxation into the sub-tree. There are three possible cases:

- If at least k vertices are found, we stop and record the result as a directed tree $\bar{F}_j$;
- If the search encounters a vertex that already belongs to an existing $\bar{F}_j$ (i.e., it overlaps with an existing sub-tree), we stop and merge the current explored sub-tree with that $\bar{F}_j$;
- Otherwise the search fails to reach k vertices from x , we record it as an arborescence W_j .

Finally we partition each $\bar{F}_j$ into $\Theta(k)$ sized edge-disjoint sub-trees in linear time as described in Appendix A.1.

An important observation is that, under a full execution, if the search fails to reach k vertices from x , we no longer need to initiate any further search from x at current value $d[x]$ with bound B .

- If x is currently complete, every vertex v with $\text{dis}(v) < B$ whose shortest path visits x is already reached and updated;
- Otherwise, the shortest path of x passes through some other complete vertex in S . We can rely on that vertex instead of x for subsequent shortest path relaxations.

Therefore, x is no longer useful for relaxation; unless $d[x]$ is updated by another relaxation and then x reappears in S in a subsequent call. Intuitively, each search from x failing to reach k vertices is attributed to one of x 's incoming edges, so it occurs at most δ times. This statement is formally proved in Lemma 3.9.

¹The algorithm also works when $\delta > \log k$. Setting $\delta = \Theta(\min\{\frac{m}{n}, \log \log n\})$ optimizes the time bound, so we require $\delta \leq \log k$ to simplify the analysis of time complexity.

Algorithm 2: Finding Pivots

Input: Upper bound B , vertex set S **Output:** Vertex sets $\{P_j\}_{j=1}^p, Q, W$

```
1 Algorithm FindPivots(upper bound  $B$ , vertex set  $S$ ):
2   forall  $x \in S$  do
3     if  $x$  is in any existing  $\bar{F}_j$  then Skip  $x$  and proceed to the next vertex in  $S$ 
4     Initialize Fibonacci heap  $H \leftarrow \{\langle x, d[x] \rangle\}$  and subgraph  $K \leftarrow \{x\}$ 
5     while  $H.\text{NON-EMPTY}()$  and  $|K| < k$  do // Local Dijkstra search from  $x$ 
6        $u \leftarrow H.\text{EXTRACT-MIN}()$ 
7       forall edge  $(u, v)$  do // Recall  $\delta \leq k$ 
8         if Relax( $u, v, B$ ) then
9           Add vertex  $v$  and edge  $(u, v)$  into  $K$ 
10          if  $v$  is in some existing  $\bar{F}_j$  then
11             $\bar{F}_j \leftarrow \bar{F}_j \cup K$ 
12            Quit the while-loop, skip  $x$  and proceed to the next vertex in  $S$ 
13          else if  $v$  is in  $H$  then
14            Remove old incoming edge of  $v$  from  $K$ 
15             $H.\text{DECREASE-KEY}(v, d[v])$ 
16          else  $H.\text{INSERT}(v, d[v])$ 
17      if  $|K| \geq k$  then Report  $K$  as a new directed tree  $\bar{F}_j$ 
18      else Report  $K$  as a new arborescence  $W_j$ 
19   $W \leftarrow \bigcup_j W_j$ ;  $Q \leftarrow \{\text{roots of } W_j\}$  //  $|W| \leq O(k|Q|)$ 
20  Partition  $\{\bar{F}_j\}$  into  $\Theta(k)$  sized edge-disjoint sub-trees  $\{F_j\}_{j=1}^p$  // See Lemma A.1
21  for  $j \leftarrow 1$  to  $p$  do
22     $P_j \leftarrow \{x \in (S \setminus Q) \cap F_j : \forall j' < j, x \notin F_{j'}\}$  //  $j$  minimality for disjointness
23  return  $\{P_j\}_{j=1}^p, Q, W$ 
```

Lemma 3.2 (Finding Pivots). Suppose $\langle \emptyset, S \rangle$ is a frontier for $\tilde{U} = \tilde{U}(B, S)$, FindPivots(B, S) (Algorithm 2) returns subsets $\{P_j\}_{j=1}^p$ of S each of size $O(k)$, another subset $Q \subseteq S$ and a set $W \subseteq \tilde{U}$ of size $O(k|Q|)$, in $O((p + |Q|)k(\delta + \log k)) = O((p + |Q|)k \log k)$ time. After running Algorithm 2, $\langle W, \bigcup_{j=1}^p P_j \rangle$ is a frontier for $\tilde{U}$.

Proof. We only need to prove that $\langle W, \bigcup_{j=1}^p P_j \rangle$ is a frontier for $\tilde{U}$.

For any $v \in \tilde{U}$, its shortest path visits some complete $u \in S$. If u is contained in some P_j , we are already done; otherwise the sub-tree searched from u contains less than k vertices. Since v lies in such a sub-tree, v is included into W and becomes complete.

As for the running time, the local Dijkstra search takes $O(1)$ time per processed edge, and $O(\log k)$ time per processed vertex as promised by Fibonacci heap [FT87]; by Lemma A.1, the partition step takes $O(1)$ time per edge. Every processed edge is incident to a vertex in some $\bar{F}_j$ or some W_j , so there are most $O((p + |Q|)k\delta)$ of them. Every processed vertex evokes heap insertion only once as a vertex of some $\bar{F}_j$, or possibly many times as a vertex in several W_j 's, which results in $O((p + |Q|)k \log k)$ time. Recall that $\delta \leq \log k$, so the total time is $O((p + |Q|)k \log k)$. $\square$

Remark 3.3. We also note that:

- Each P_j is connected by a directed tree F_j , where F_j 's are edge-disjoint, and vertices of F_j are in $\tilde{U}$.
- $\{P_j\}_{j=1}^p$ and Q are disjoint, and their union is S . Moreover, $p \leq \min\{|S|, |\tilde{U}|/k\}$.
- W is a union of $|Q|$ arborescences $\{W_j\}_{j=1}^{|Q|}$; Q are their roots.
- However, $\bigcup_{j=1}^p F_j$ and the arborescences in W are not necessarily disjoint.

3.2 Data Structure

[DMM⁺25] designed a data structure to support the operations required for partially sorting the “frontiers”. In this section we present a simpler data structure with a slight improvement. We implement by a self-balanced binary search tree (BST) of M -sized blocks: blocks sorted by the BST, but items inside a block unsorted. Requiring $\log(N/M) \leq M$, an $O(\log(N/M))$ time BST operation is amortized to $O(1)$ per item in blocks, which enables us to perform operations except insertions in amortized linear time.

Lemma 3.4 (Data Structure). *Given at most N key/value pairs involved, a parameter M , and an upper bound B , there exists a data structure $\mathcal{D}$ that supports the following operations:*

Insert *Insert a key/value pair. If the key already exists, keep the one with smaller value.*

Merge *For another data structure $\mathcal{D}'$ with smaller parameter $M' (< M/3)$ as specified in this lemma, such that all the values in $\mathcal{D}'$ are smaller than all the values in $\mathcal{D}$, insert all pairs of $\mathcal{D}'$ into $\mathcal{D}$ (keep the smaller value for duplicate keys).*

Pull *Return a subset S' of keys where $|S'| \leq M$ associated with the smallest $|S'|$ values and an upper bound x that separates S' from the remaining values in $\mathcal{D}$. Specifically, if there are no remaining values, $x = B$; otherwise, $|S'| = M$.*

If $M = 1$, INSERT and PULL take $O(\log N)$ time with MERGE unsupported, for base case only (Section 3.7).

If $M > 1$, we require $M \geq \log(N/M)$. INSERT takes $O(\log(N/M))$ amortized time; other operations are linear: MERGE takes $O(|\mathcal{D}'|)$ time and PULL takes $O(|S'|)$ time.

The proof of Lemma 3.4 is given in Appendix A.2.

3.3 Bounded Multi-Source Shortest Paths

In this section we are ready to present our main BMSSP algorithm, see Algorithm 3.

The base case ($l = 0$) can be solved using Dijkstra's algorithm as in Section 3.7. Now we assume $l > 0$.

We first initialize data structure $\mathcal{D}$ with parameter $M = t2^{(l-1)t}$ to hold the vertices we will scan.

Then we run FindPivots of Lemma 3.2 to reorganize S into disjoint subsets $\{P_j\}_{j=1}^p$ and Q of S , with a set W . For each P_j we find $p_j = \arg \min_{x \in P_j} d[x]$ the minimum vertex in it. We will update p_j accordingly such that p_j always lower bounds necessary² vertices in P_j .

Set $B' \leftarrow B'_0 \stackrel{\text{def}}{=} \min\{B, \{d[p_j] : 1 \leq j \leq p\}\}$; $U \leftarrow \emptyset$. B' actually marks the progress of our algorithm. U will accumulate batches of complete vertices. Set $J \leftarrow \emptyset$; J holds the set of j such that p_j needs to be re-selected. The rest of algorithm repeats the following steps for multiple rounds until $\mathcal{D}$ becomes empty or $|U| > t^{3/2l}$, where during the i -th iteration ($i \geq 1$), we:

²By “necessary” we mean a vertex in P_j whose shortest path does not visit any complete vertex in $\mathcal{D}$; because if it does visit, such a vertex need not to be tracked. For better readability, we recommend that readers first form a high-level picture of the algorithm and then return to this definition. A formal proof that the algorithm preserves this property appears in Lemma 3.7.

Algorithm 3: BMSSP

Input: Upper bound B , vertex set $S(|S| \leq t^2 2^{lt})$, layer l
Output: New bound $B'(\leq B)$; vertex set $U(|U| = O(t^3 2^{lt}))$; data structure $\mathcal{D}$

1 **Algorithm** BMSSP(*upper bound B , vertex set S , layer l*):

2 **if** $l = 0$ **then** Operate and return as Algorithm 4 (Base Case)

3 Initialize data structure $\mathcal{D}$ parameterized by $M = t^2 2^{(l-1)t}$

4 $\{P_j\}_{j=1}^p, Q, W \leftarrow \text{FindPivots}(B, S)$

5 **for** $j \leftarrow 1$ **to** p **do**

6 $p_j \leftarrow \arg \min_{x \in P_j} d[x]$

7 $\mathcal{D}.\text{INSERT}(\langle p_j, d[p_j] \rangle)$

8 $B' \leftarrow B'_0 \stackrel{\text{def}}{=} d_B[\{p_j : 1 \leq j \leq p\}]; U \leftarrow \emptyset; i \leftarrow 1; J \leftarrow \emptyset$

9 **while** $|U| \leq t^3 2^{lt}$ **and** $\mathcal{D}$ is non-empty **do**

10 $S_i, B_i \leftarrow \mathcal{D}.\text{PULL}()$

11 **forall** $x \in S_i$ **do** // $|S_i|$ expands by at most k times.

12 **if** $x = p_j$ for some j **then**

13 $S_i \leftarrow S_i \cup \{v \in P_j : d[v] < B_i\}$

14 $B'_i, U_i, \mathcal{D}_i \leftarrow \text{BMSSP}(B_i, S_i, l-1)$

15 $\mathcal{D}.\text{MERGE}(\mathcal{D}_i)$

16 **forall** $u \in U_i$ **do**

17 **if** $u \in P_j$ for some j **then**

18 Remove u from P_j

19 **if** $u = p_j$ **and** $P_j \neq \emptyset$ **then**

20 Add j into J

21 **forall** $u \in U_i$ **do**

22 **forall** edge (u, v) **do** // u is complete, $d[u] = \text{dis}(u)$

23 **if** $\text{Relax}(u, v, B)$ **and** $d[u] + w_{uv} \geq B_i$ **then**

24 $\mathcal{D}.\text{INSERT}(\langle v, d[v] \rangle)$

25 **if** $v \in P_j$ for some j **and** $j \notin J$ **and** $d[p_j] > d[v]$ **then**

26 $p_j \leftarrow v$

27 **forall** $j \in J$ **do**

28 $p_j \leftarrow \arg \min_{x \in P_j} d[x]$

29 $\mathcal{D}.\text{INSERT}(\langle p_j, d[p_j] \rangle)$

30 $B' \leftarrow B'_i; U \leftarrow U \cup U_i; i \leftarrow i + 1; J \leftarrow \emptyset$

31 **forall** $x \in S$ **do**

32 **if** $d[x] \in [B', B)$ **then**

33 $\mathcal{D}.\text{INSERT}(\langle x, d[x] \rangle)$

34 **forall** $u \in W' \stackrel{\text{def}}{=} \{x \in W \setminus U : d[x] < B'\}$ **do**

35 **forall** edge (u, v) **do** // u is complete, $d[u] = \text{dis}(u)$

36 **if** $\text{Relax}(u, v, B)$ **and** $d[u] + w_{uv} \geq B'$ **then**

37 $\mathcal{D}.\text{INSERT}(\langle v, d[v] \rangle)$

38 **return** $B', U \leftarrow U \cup W', \mathcal{D}$

- Pull from $\mathcal{D}$ a subset S_i of keys with smallest values in $\mathcal{D}$ upper bounded by B_i (also a lower bound for remaining vertices in $\mathcal{D}$). Scan $x \in S_i$; if $x = p_j$ for some j , add $\{v \in P_j : d[v] < B_i\}$ into S_i (by brute-force search);
- Recursively call $\text{BMSSP}(B_i, S_i, l - 1)$ and collect its output $B'_i, U_i, \mathcal{D}_i$;
- Merge vertices of $\mathcal{D}_i$ into $\mathcal{D}$;
- For each $u \in U_i$, if $u \in P_j$ for some j , remove u from P_j ; and if $u = p_j$, also add j into J .
- For each $u \in U_i$, relax every edge (u, v) such that $d[u] + w_{uv} \in [B_i, B]$; if the relaxation is valid, insert $\langle v, d[v] \rangle$ into $\mathcal{D}$; if $v \in P_j$ for some $j \notin J$, also update p_j to the smaller one between v and p_j ;
- For each non-empty P_j from $j \in J$, re-select p_j to the minimum vertex in P_j by brute-force search and insert it into $\mathcal{D}$;
- Update U to include U_i and $B' \leftarrow B'_i$; Reset $J \leftarrow \emptyset$;
- If $\mathcal{D}$ becomes empty, then this is a full execution and we quit the loop;
- If $|U| > t^3 2^{lt}$, this is a partial execution where we encountered a large workload. Also quit the loop.

After quitting the loop, we operate the following finalizing operations:

- For every $x \in S$ such that $d[x] \in [B', B]$, insert $\langle x, d[x] \rangle$ into $\mathcal{D}$.
- For every edge (u, v) from every $u \in W' \stackrel{\text{def}}{=} \{x \in W \setminus U : d[x] < B'\}$ such that $d[u] + w_{uv} \in [B', B]$, relax it; if the relaxation is valid, insert $\langle v, d[v] \rangle$ into $\mathcal{D}$.
- $U \leftarrow U \cup W'$, i.e., add every vertex $u \in W'$ into U .

3.4 Observations and Discussions on the Algorithm

We first present some informal explanations to give reader an intuitive overview on the algorithm, and in subsequent subsections we will provide more formal proofs on correctness and running time.

Recursion. We first think of the recursion tree $\mathcal{T}$ of Algorithm 3 where each node X denotes a call of BMSSP algorithm. We subscript X to all the parameters to denote those used in the call of X : l_X, B_X, S_X for input, $B'_X, U_X, \mathcal{D}_X$ for output, $\{P_{X,j}\}_{j=1}^{p_X}, Q_X, W_X$ for output of FindPivots (Line 4). Also denote $W'_X = \{x \in W_X \setminus U_X : d(x) < B'_X\}$, $P_X = \bigcup_{j=1}^{p_X} P_{X,j}$, and $\tilde{U}_X = \tilde{U}(B_X, S_X)$ as above. Then we have the following properties:

1. In the root R of $\mathcal{T}$, $U_R = V$;
2. Think of the construction of S_X in the caller Z of X : when pulled (Line 10), $|S_X| \leq M_Z = t^{2(l_Z-1)t} = t^{2l_X t}$. Adding $\{v \in P_{Z,j} : d[v] < B_X, p_{Z,j} \in S_X\}$ into S_X (Line 13) expands $|S_X|$ by at most $O(k)$ times.

Therefore $|S_X| \leq t^{2l_X t}$, so the depth of tree $\mathcal{T}$ is at most $(\log n)/t$;

3. We break when $|U_X| > t^3 2^{l_X t} \geq t|S_X|$. Intuitively, $|U_X|$ grow slowly enough so we still have $|U_X| = O(t^3 2^{l_X t})$ (see Lemma 3.8).

4. If X is a full execution, then $\tilde{U}_X = U_X$. If X is a partial execution, then $|U_X| > t|S_X|$. In both cases, by Remark 3.3, $p_X \leq \min\{|\tilde{U}_X|/k, |S_X|\} \leq |U_X|/k$.
5. Suppose $Y_1, Y_2, \dots, Y_f$ are the sub-calls invoked by X , i.e., the children of X in $\mathcal{T}$. Then we have:
 - $U_{Y_1}, U_{Y_2}, \dots, U_{Y_f}$ are disjoint. For $i < j$, $\text{dis}()$ of vertices in U_{Y_i} are smaller than that of U_{Y_j} .
 - U_X is a disjoint union of W'_X and $U_{Y_1}, U_{Y_2}, \dots, U_{Y_f}$.
 - The U_X 's for all nodes at the same layer in $\mathcal{T}$ are disjoint, and total size of all these $|U_X|$ in each layer is $O(|V|)$.
6. For any vertex u , there is only at most one call in each layer whose U contains it. Thus, the calls X whose U_X contains u forms a path in $\mathcal{T}$: $R = X_0, X_1, \dots, X_q$ such that:
 - For $0 \leq i < q$, X_{i+1} is a sub-call of X_i ;
 - For $0 \leq i < q$, $u \in U_{X_i}$; relaxations from u (Line 24) lie in interval $[B_{X_{i+1}}, B_{X_i})$;
 - For the last call X_q containing u , either $u \in W'_{X_q}$ and relaxations from u (Line 37) lie in interval $[B'_{X_q}, B_{X_q})$; or $u \in U_{X_q}$ where X_q is a base case call and relaxations from u (Line 7) lie in interval $[0, B_{X_q})$;
 - So the intervals for $X_0, \dots, X_q$ are disjoint, and any edge can be validly relaxed at most once, in Line 24, Line 37, or a base case call.

Correctness. Base case is solved using Dijkstra's algorithm and is correct. Below we assume $l > 0$.

The framework of Section 2.5 helps understand how our algorithm works and verify correctness. In a call of BMSSP, $\tilde{U} = \tilde{U}(B, S)$ is the set that contains all vertex v with $\text{dis}(v) < B$ whose shortest path visits some vertex in S , and is our expected target. Then BMSSP should return $U = \tilde{U}$ in a successful execution, or $U = \{u \in \tilde{U} : \text{dis}(u) < B'\}$ in a partial execution, with all vertices in U complete.

At the beginning, we reorganize S as $\Theta(k)$ -sized chunks $\{P_{X,j}\}_{j=1}^{p_X}$ to insert into $\mathcal{D}$, with some vertices complete and added into W . Lemma 3.2 ensures that the shortest path of remaining incomplete vertices in $\tilde{U}$ visit some complete vertex in P , i.e., we push the frontier to $\langle W, P \rangle$.

We do not insert all of P , but only the minimum vertex ("pivot") of each P_j , i.e., $p_j = \arg \min_{x \in P_j} d[x]$ into $\mathcal{D}$. Then the shortest paths of remaining incomplete vertices in $\tilde{U}$ visit some complete vertex in $\mathcal{D} \cup P$. For any bound $B_i \leq B$, if $\text{dis}(v) < B_i$, the shortest path of v must also visit some complete vertex in $u \in \mathcal{D} \cup P$ with $\text{dis}(u) < B_i$. When we pull S_i a subset from $\mathcal{D}$, the pivots $p_j \in S_i$ helps augment S to contain necessary vertices bounded by B_i from $\mathcal{D} \cup P$. Then we call sub-procedure BMSSP on B_i and S_i .

By the inductive hypothesis, each recursive call on Line 14 of Algorithm 3 returns U_i and $\mathcal{D}_i$, vertices in U_i are complete, and $\mathcal{D}_i$ contains vertices remaining in S_i , and vertices validly relaxed from U_i with distance less than B_i . What we do actually matches Section 2.5: relax edge (u, v) from $u \in U_i$, if $B'_i \leq \text{dis}(u) + w_{uv} < B_i$, v is already in $\mathcal{D}_i$; otherwise, v is relaxed and inserted to $\mathcal{D}$ directly. Once again remaining incomplete vertices in $\tilde{U}$ now visits some complete vertex in $\mathcal{D} \cup P$.

If p_j is in U_i and removed, we re-select a new minimum p_j and insert it into $\mathcal{D}$. It becomes complicated when p_j is not removed while still $d[v]$ gets decreased for some other $v \in P_j$, because we cannot afford to re-select a new p_j . In this case we update p_j to lower bound only a subset $\hat{P}_j \subseteq P_j$, the set of vertices in P_j whose shortest path do not visit any complete vertex in $\mathcal{D}$ — we do not explicitly compute $\hat{P}_j$, merely introducing it for explanation. Because we may rely on $\mathcal{D}$, we do not need to keep track of vertices not in $\hat{P}_j$. This observation will be explained further in Lemma 3.7.

After we quit the loop, if this is a full execution ($B' = B$), we return $\mathcal{D} = \emptyset$. If this is a partial execution, $\mathcal{D}$ already includes all vertex v relaxed from all $u \in U_i$ which satisfies $B' \leq \text{dis}(u) + w_{uv} < B$. Then we first

add back remaining vertices in S , whose distance labels are in $[B', B)$; they are premature vertices and need to be added back to frontier again in the upper layer. We also need to relax from $W' = \{x \in W \setminus U : d[x] < B'\}$ which includes vertices whose shortest paths visits complete vertices in Q .

Running time The running time is dominated by calls of `FindPivots`, the overheads inside the data structures $\mathcal{D}$, selecting $p_j = \arg \min_{x \in P_j} d[x]$ from P_j 's and picking $\{x \in P_j : d[x] < B_i\}$ for each $p_j \in S_i$. First we make an observation:

Observation 3.5. *Any edge (u, v) can only enter the execution of Line 24 once in the whole algorithm.*

Proof. This can only happen when u becomes complete in a call X which is a child of current call Y in $\mathcal{T}$, so there is at most once in one layer. Other call Z where U_Z contains u must be ancestor or descendant of X . Also in the current call Y we need $B_X \leq d[u] + w_{uv} < B_Y$ to make Line 24 happen, but for descendant Z of X (including X itself) in $\mathcal{T}$, $B_Z \leq B_X$, so it is impossible that $d[u] + w_{uv} < B_Z$. So Line 24 will not happen in lower layers, thus only once in the whole recursion. $\square$

By Item 4 and Lemma 3.2 above, $p_X \leq |U_X|/k$, so the total running time of `FindPivots` over one layer of $\mathcal{T}$ is $O(|V| \log t + \sum_{X \text{ in one layer of } \mathcal{T}} |Q_X|t)$, since $t = k \log t > k \log k$. To estimate $\sum_{X \in \mathcal{T}} |Q_X|$:

- If X is a partial execution, $|Q_X| \leq |S_X| < |U_X|/t$.
- If X is a full execution, for any $v \in Q_X$, $v \in U_X$. For v 's latest relaxed edge (u, v) at the beginning of call X (Line 24, 37):
 - Since $v \in S_X$, u is not in U_X .
 - If there is another full execution Y with $v \in Q_Y$, X and Y is of ancestor-descendant relationship.
 - We do not put vertices in Q_X in $\mathcal{D}_X$ directly, thus if $v \in Q_Y$ for a descendant Y of X which is also a full execution, its latest relaxation edge must be different from (u, v) .

Thus the number of appearances of v in Q_X in all full executions X is bounded by δ , the degree bound of v . Therefore $\sum_{X \in \mathcal{T}} |Q_X| = O(|V|(\delta + l/t))$.

The total time for `FindPivots` is $O(|V|l \log t + t \sum_{X \in \mathcal{T}} |Q_X|) = O(mt + (m \log n \log t)/(t\delta))$. (Recall the number of vertices is $O(m/\delta)$.)

For the data structure $\mathcal{D}$ in a call of `BMSSP`, the total running time is dominated by `INSERT` as all other operations take linear time. It is clear that vertices added into $\mathcal{D}_X$ include a subset of S_X , and some out-going neighbors of U_X , so the size of $\mathcal{D}_X$ is bounded by $|S_X| + |U_X|\delta \leq t^4 2^{lt}$, so each insertion takes time $O(t)$.

- For pivots p_j to $\mathcal{D}$ in Line 7, $p_X \leq |U_X|/k$, so the time is $O(|U_X|t/k) = O(|U_X| \log t)$.
- For vertices inserted into $\mathcal{D}$ in Line 33, they are non-zero only when X is a partial execution ($B'_X < B_X$), such that $|S_X| \leq |U_X|/t$, so the time is $O(t|S_X|) \leq O(|U_X|)$.
- By Observation 3.5, a vertex v can only be inserted to $\mathcal{D}$ by edge (u, v) in Line 24 once. Also a vertex u can only be in W' once as $W' \subseteq U \setminus (\bigcup U_i)$, so each edge (u, v) can only be relaxed once in Line 37.

Therefore, the total time for Line 7, Line 33, Line 24 and Line 37 is $O(mt + (m \log t \log n)/(t\delta))$.

Next we consider the time of picking (Line 13), re-selection and insertion (Line 29) of X . Similar to Item 5, suppose $Y_1, Y_2, \dots, Y_f$ are sub-calls of X .

- Picking $\{x \in P_j : d[x] < B_i\}$ (Line 13) takes time $O(k)$ per $p_j \in S_{Y_i}$.

- If Y_i is a partial execution, time is bounded by $O(k|S_{Y_i}|) = O(|U_{Y_i}|)$;
- If Y_i is a full execution, for any $p_j \in S_{Y_i}$, $p_j \in U_{Y_i}$ and is removed from P_j , so P_j needs to re-select and insert a new p_j which takes $O(t)$ time. We amortize this $O(k)$ to that operation as $k \leq t$. (If P_j becomes empty, this happens at most one time for each P_j ; these contribute to at most $O(kp) = O(|U_X|)$.)

So this step takes time $O(|U_X|)$ for X , and summing over all calls we get $O((m \log n)/(t\delta))$.

- Re-selection and insertion (Line 29) take $O(t)$ time per p_j . For a call X ,
 - If X is a partial execution, time is bounded by $O(|S_X|t) = O(|U_X|)$.
 - If X is a full execution, for each re-selection and insertion (Line 29) of each $P_{X,j}$, we want to establish an injection from them to one tree vertex of $F_{X,j}$ in each U_{Y_i} . The number of tree vertices in different U_{Y_i} 's is no more than one plus the number of “cross-level” edges in $F_{X,j}$, which are the edges connecting two different U_{Y_i} 's, which are not used by full sub-calls (full Y_i only uses edges connecting two vertices within U_{Y_i} , nor callers of X (X is a full execution), such that these edges are globally uniquely consumed by X .

Instead of each $p_{X,j}$ in Line 29 itself, we think of its former pivot of $P_{X,j}$, $p'_{X,j}$, which is in U_{Y_i} and was removed from D in i -th round, so j is put into J . Each $p_{X,j}$ uniquely corresponds to $p'_{X,j}$, and each $p'_{X,j}$ belongs to some unique U_{Y_i} : there is at most one pivot $p'_{X,j}$ removed from D in each U_{Y_i} .

Therefore, we amortize the time of re-selection and insertion (Line 29) for $P_{X,j}$ to edges, such that each edge is amortized at most once in the whole algorithm. (Except once for each $P_{X,j}$ which sums up to $O(p_X t) = O(|U_X| \log t)$.)

Summing over all calls we get $O((m \log n \log t)/(t\delta) + mt)$.

In total, the algorithm runs in $O(m(t + \frac{\log n \log t}{\delta t}))$ time, dominated by FindPivots.

3.5 Correctness Analysis of BMSSP

In this section we verify Algorithm 3 fits the Dijkstra-like algorithm framework proposed in Section 2.5.

Lemma 3.6 explains why we can invoke a recursive call on a smallest set extracted from the frontier.

Lemma 3.6 (Frontier Split). *Suppose at some moment, $\langle X, Y \rangle$ is a frontier for $\tilde{U} = \tilde{U}(B, S)$. For a smaller $B_s \leq B$, suppose $\{x \in Y : d[x] < B_s\} \subseteq Y_s \subseteq \tilde{U}$. Then $\langle \emptyset, Y_s \rangle$ is a frontier for $\tilde{U}(B_s, Y_s)$.*

Proof. For any $v \in \tilde{U}(B_s, Y_s)$, its shortest path visits some $y \in Y_s$. If y is complete, then we are done. If y is incomplete, as $y \in Y_s \subseteq \tilde{U}$, the shortest path of y , as a prefix of the shortest path of v , must visit some complete $y' \in Y$. Then the shortest path of v visits y' and $y' \in Y_s$ as $d[y'] = \text{dis}(y') \leq \text{dis}(v) < B_s$. $\square$

We formally define the set R mentioned in Section 2.5. For a *complete* set Z and any bound B , after relaxing the outgoing edges from Z (not exceeding B), denote $R(B, Z)$ as the set of out-neighbors v of Z but not in Z with distance less than B , and relaxations from some vertex $u \in Z$ to v is valid. Equivalently, after relaxing outgoing edges from Z , $R(B, Z) = \{v : \exists u \in Z, (u, v) \in E, v \notin Z, d[v] = d[u] + w_{uv} < B\}$.

Denote $\hat{P}_j$ the set of vertex in P_j whose shortest path does not visit any complete vertex in $\mathcal{D}$. Also denote $P = \bigcup_{j=1}^p P_j$; $\hat{P} = \bigcup_{j=1}^p \hat{P}_j$. We define them to be up-to-date through algorithm progress. Clearly, any shortest path visits some complete vertex in $\mathcal{D} \cup P$, if and only if it visits some complete vertex in $\mathcal{D} \cup \hat{P}$.

We introduce $\hat{P}_j$ because for some $v \in P_j$, after partial i -th sub-call, $d[v]$ may decrease but v is not in $\mathcal{D}_i$ or U_i such that we cannot maintain p_j to minimum among P_j . This is because in a partial execution as we

will see in Lemma 3.7, $\mathcal{D}$ contains $S \setminus U$ and only a thin shell of out-going neighbors from U , while vertices in $\tilde{U}$ two or more hops away from U may be modified but not returned. Fortunately, the shortest path of such vertices visit some complete vertex in $\mathcal{D}$ such that we do not need to track them from P_j .

We denote P_{ini} the initial value of P , i.e., $\bigcup_{j=1}^p P_j$ returned by FindPivots.

Lemma 3.7 (Correctness of BMSSP, focusing on frontier and completeness). *Given an upper bound B , a vertex set S , suppose $\langle \emptyset, S \rangle$ is a frontier for $\tilde{U} = \tilde{U}(B, S)$. After running Algorithm 3:*

1. $\langle U, \mathcal{D} \rangle$ is a frontier for $\tilde{U}$;
2. $\max_{x \in U} \text{dis}(x) < B' \leq d_B[\mathcal{D}]$, $U = \tilde{U}(B', S)$ and is complete;
3. $\mathcal{D}$ contains $S \setminus U$ and every out-neighbor v of U but not in U validly relaxed from $u \in U$ with $d[u] + w_{uv} = d[v] < B$. Namely, $\mathcal{D} \supseteq (S \setminus U) \cup R(B, U)$.

Proof. We prove by induction the following three propositions: at the beginning of i -th iteration (immediately before Line 10, or equivalently, immediately after Line 30, at the end of $(i - 1)$ -th iteration):

1. $\langle U, \mathcal{D} \cup P \rangle$ is a frontier for $\tilde{U}_P \stackrel{\text{def}}{=} \tilde{U}(B, P_{ini})$;
2. $d_B[\mathcal{D} \cup P] \geq B' (= B'_{i-1})$;
3. For each non-empty P_j , $d[p_j] \leq d_B[\hat{P}_j]$.

Clearly they hold for the first iteration by Lemma 3.2 and Observation 2.1. Now suppose they hold at the beginning of i -th iteration, we prove that they hold at the end of the i -th iteration.

We first show that: the prerequisite conditions of Lemma 3.7 are met for the i -th sub-call (Line 14).

Since $\langle U, \mathcal{D} \cup P \rangle$ is a frontier for $\tilde{U}_P$, $\langle U, \mathcal{D} \cup \hat{P} \rangle$ is also a frontier for $\tilde{U}_P$. $\mathcal{D}$ pulls S_i bounded by B_i whereas B_i lower bounds remaining vertices in $\mathcal{D}$ (Line 10). Picking vertices in P_j (Line 13) adds every $v \in \hat{P}$ with $d[v] < B_i$ into S_i , because $d[p_j]$ lower bounds each $\hat{P}_j$. Thus, S_i contains every $v \in \mathcal{D} \cup \hat{P}$ with $d[v] < B_i$. By Lemma 3.6, $\langle \emptyset, S_i \rangle$ is a frontier for $\tilde{U}(B_i, S_i)$.

Clearly base case satisfies Lemma 3.7. By induction, the sub-calls satisfy the conclusions of Lemma 3.7.

Subsequent operations in i -th iteration is indeed using complete set U_i to update its frontier $\langle U, \mathcal{D} \cup P \rangle$ as described in Section 2.5:

1. U obtains vertices in a complete set U_i (Line 30);
2. $\mathcal{D} \cup P$ loses *only* vertices in U_i :
 - P loses vertices U_i (Line 18);
 - U_i is not in $\mathcal{D}$, no need to remove: $\forall u \in U_i, d[u] = \text{dis}(u) < B' \leq d_B[\mathcal{D}]$.
 - $S_i \setminus U_i$ was removed from $\mathcal{D} \cup P$, but merged back into $\mathcal{D}$ again as $\mathcal{D}_i \supseteq S_i \setminus U_i$ (Line 15);
3. $\mathcal{D} \cup P$ receives $R(B, U_i)$, the set of out-neighbors v of Z but not in Z validly relaxed from complete vertex $u \in U_i$ with $d[u] + w_{uv} = d[v] < B$:
 - If $d[u] + w_{uv} < B_i$, v is included by $\mathcal{D}_i$ and merged into $\mathcal{D}$ (Line 15);
 - If $d[u] + w_{uv} \in [B_i, B)$, v is directly inserted into $\mathcal{D}$ (Line 24);

Therefore, at the end of the i -th iteration, $\langle U, \mathcal{D} \cup P \rangle$ is a new frontier for $\tilde{U}_P$.

Now we verify that: at the end of the i -th iteration, $d_B[\mathcal{D} \cup P] \geq B' (= B'_i)$:

- For $\mathcal{D}$, after PULL (Line 10), $d_B[\mathcal{D}] \geq B_i$. For any v added into $\mathcal{D}$, $d[v] \geq B'_i$. Finally, $d_B[\mathcal{D}] \geq B'_i$;
- For $\widehat{P}$, we picked out every $v \in \widehat{P}$ with $d[v] < B_i$ (Line 13), so for the remaining vertices, $d_B[\widehat{P}] \geq B'_i$.
- For $P \setminus \widehat{P}$, their shortest paths visit some complete vertices in $\mathcal{D} \cup \widehat{P}$, $d_B[P \setminus \widehat{P}] \geq d_B[\mathcal{D} \cup \widehat{P}] \geq B'_i$.

Now we verify that: at the end of the i -th iteration, for each non-empty P_j , $d[p_j] \leq d_B[\widehat{P}_j]$.

1. If a new p_j is newly re-selected from P_j (Line 29), clearly $p_j = d_B[P_j] \leq d_B[\widehat{P}_j]$.
2. Otherwise, p_j is not in U_i and not removed from P . For any $v \in \widehat{P}_j$, at the beginning, $d[p_j] \leq d[v]$.
 - If $d[v]$ is modified by i -th sub-call, then $v \in \widetilde{U}_i = \widetilde{U}(B_i, S_i)$. As $\langle U_i, \mathcal{D}_i \rangle$ is a frontier of $\widetilde{U}_i$:
 - If $v \in U_i$, v is removed from P_j , so $v \notin \widehat{P}_j$;
 - If the shortest path of v visits some complete vertex in $\mathcal{D}_i$ merged into $\mathcal{D}$, $v \notin \widehat{P}_j$.
 - If $d[v]$ is modified by valid relaxation from $u \in U_i$ where $d[u] + w_{uv} = d[v] \geq B_i$ (Line 24), then p_j is chosen to be the smaller one (Line 26): $d[p_j] \leq d[v]$;
 - Otherwise, $d[v]$ does not change over i -th iteration, at the end, $d[p_j] \leq d[v]$.

The induction proof of three propositions are complete. Finally, we proceed to prove Lemma 3.7.

Suppose in total, there are f iterations. At the end of the f -th iteration, we have that: $\langle U, \mathcal{D} \cup P \rangle$ is a frontier for $\widetilde{U}_P = \widetilde{U}(B, P_{ini})$; $d_B[\mathcal{D} \cup P] \geq B'$. We verify the following statements sequentially:

- At the end of the f -th iteration, $W' = \{x \in W \setminus U : d[x] < B'\}$ is complete.
If $v \in W' \subseteq W$ is incomplete, because $\langle W, P_{ini} \rangle$ is a frontier for $\widetilde{U}$, v visits some complete vertex in P_{ini} . Then $v \in \widetilde{U}_P$, so the shortest path of v visits some complete vertex $u \in \mathcal{D} \cup P$, such that $\text{dis}(v) \geq \text{dis}(u) = d[u] \geq d_B[\mathcal{D} \cup P] \geq B'$, but this violates with $d[v] < B'_i$ by its definition.

Thus W' is complete.

- At the end of the f -th iteration, for any vertex v in $\widetilde{U}$, v is complete, or the shortest path of v visits some complete vertex in $\mathcal{D} \cup P$; or the shortest path of v visits some complete vertex in $q \in Q$, such that $d[q] = \text{dis}(q) < B'$ and $q \in W'$, or $d[q] = \text{dis}(q) \in [B', B)$.

In other words: $\langle U, \mathcal{D} \cup P \cup W' \cup Q^* \rangle$ is a frontier for $\widetilde{U}$, where $Q^* = \{x \in Q : d[x] \in [B', B)\}$.

For any $v \in \widetilde{U}$, if $v \in \widetilde{U}_P$, then v is handled by U and $\mathcal{D} \cup P$. If $v \in \widetilde{U} \setminus \widetilde{U}_P$, the shortest path of v visits some $q \in Q = S \setminus P_{ini} \subseteq W$ which has been complete at the beginning of this call. Currently $U \subseteq \widetilde{U}_P$, so $q \notin U$. Thus, $q \in W'$ or $q \in Q^*$.

- After Line 33, $\langle U, \mathcal{D} \cup W' \rangle$ is a frontier of $\widetilde{U}$.

For every vertex $v \in P$, $d[v] \geq d_B[P] \geq B'$ and is inserted back to $\mathcal{D}$ (Line 33). For every vertex $v \in Q^*$, as $d[v] \geq B'$, v is also inserted back to $\mathcal{D}$ (Line 33).

- When Algorithm 3 returns, $d_B[\mathcal{D}] \geq B'$;

Because all values inserted into $\mathcal{D}$ after the f -th iteration (after Line 30) are at least B' , $d_B[\mathcal{D}] \geq B'$.

- When Algorithm 3 returns, $\langle U, \mathcal{D} \rangle$ is a frontier of $\widetilde{U}$.

The remaining part of the algorithm (from Line 34 to Line 37) is indeed using complete set W' to update its frontier $\langle U, \mathcal{D} \cup W' \rangle$ to $\langle U, \mathcal{D} \rangle$ as described in Section 2.5: $\max_{x \in W'} d[x] < B' \leq d_B[\mathcal{D}]$ so no need to remove W' from U ; all out-neighbors of W' but not in W' with distance less than B but validly relaxed by W' , i.e., vertices in $R(B, W')$, are inserted into $\mathcal{D}$.

- When Algorithm 3 returns, U is complete;

By induction, U_i are complete; and we have shown W' is complete, so U is complete.

- When Algorithm 3 returns, $\max_{x \in U} \text{dis}(x) < B'$ and $U = \tilde{U}(B', S)$.

By induction, $\max_{x \in U} \text{dis}(x) \leq \max\{\max_{x \in U_f} \text{dis}(x), \max_{x \in W'} \text{dis}(x)\} < B'_f = B'$.

By Observation 2.1 and since $\langle U, \mathcal{D} \rangle$ is a frontier for $\tilde{U}$ such that $d_B[\mathcal{D}] \geq B'$, we have $\tilde{U}(B', S) \subseteq U$; on the other hand, since $\max_{x \in U} \text{dis}(x) < B'$, $U \subseteq \tilde{U}(B', S)$. Thus $U = \tilde{U}(B', S)$.

- When Algorithm 3 returns, $\mathcal{D}$ contains $S \setminus U$ and every out-neighbor v of U but not in U validly relaxed from $u \in U$ with $d[u] + w_{uv} = d[v] < B$, i.e., every vertex in $R(B, U)$.

For any $v \in S \subseteq \tilde{U}$, if $d[v] < B' \leq d_B[\mathcal{D}]$, then $v \in \tilde{U}(B', S) \subseteq U$.

Thus $S \setminus U \subseteq \{x \in S : d[x] \in [B', B)\} \subseteq \mathcal{D}$.

For every out-neighbor v of U but not in U validly relaxed from $u \in U$ with $d[u] + w_{uv} = d[v] < B$, by $U = \tilde{U}(B', S)$ we know that $d[v] \geq B'$; and we can verify $v \in \mathcal{D}$ by direct insertion or merge (Line 24, Line 15, Line 37). Thus, $\mathcal{D} \supseteq (S \setminus U) \cup R(B, U)$.

□

Lemma 3.8 (Correctness of BMSSP, focusing on size constraints). *Under the same conditions of Lemma 3.7, $|U| = O(t^3 2^{lt})$. Especially, for partial execution, $|U| = \Theta(t^3 2^{lt})$. Every insertion in Algorithm 3 takes $O(t)$ time.*

For U_i 's returned by the sub-calls under one single call of BMSSP, they are disjoint. As a result, U is a disjoint union of U_i 's and W' .

Proof. $|U|$ constraints follow from the while-condition of Algorithm 3 and induction: $t^3 2^{lt} + t^3 2^{(l-1)t} = \Theta(t^3 2^{lt})$. For insertion time, as $|U| \leq O(t^3 2^{lt})$, the number of vertices added into $\mathcal{D}$ is bounded by $|S| + |U|\delta \leq O(t^4 2^{lt})$ ($\delta \leq k \leq t$). With $M = t^2 2^{(l-1)t}$, each insertion takes $O(\log(t^4 2^{lt}/M)) = O(t)$ time.

For U_i disjointness, we only need to show: $\{\text{dis}(v) : v \in U_i\} \subseteq [B'_{i-1}, B'_i]$. By Lemma 3.7, $\max_{u \in U_i} \text{dis}(u) < B'_i$. Also by Lemma 3.7, in the i -th iteration, $\text{dis}_B(U_i) \geq \text{dis}_B(S_i) = d_B[S_i] \geq d_B[\mathcal{D} \cup P] \geq B'_{i-1}$. □

3.6 Time analysis of BMSSP

Following [DMM⁺25], for a vertex set U and two bounds $c < d$, denote $N_{[c,d]}^+(U) = \{(u, v) \in E : u \in U, \text{dis}(u) + w_{uv} \in [c, d]\}$. Clearly $|N_{[c,d]}^+(U)| \leq \delta|U|$.

Denote $E(U) = \{(u, v) \in E : u, v \in U\}$. Clearly $|E(U)| \leq \delta|U|$.

Recall in Section 3.4 we defined the recursion tree $\mathcal{T}$ of Algorithm 3. For X a BMSSP call, we subscript X to all the parameters to denote those used in X . Denote $\mathcal{T}(X) = \{X' : X' \text{ is in the subtree rooted at } X\}$ and $\mathcal{T}^\circ(X) = \mathcal{T}(X) \setminus \{X\}$.

Lemma 3.9 (Time analysis of BMSSP). *Denote C to be a sufficiently large time constant. Under the same conditions with Lemma 3.7, a call X of BMSSP runs in time: (here $U = U_X$)*

$$C|U|(l + \log t) \log t + Ct|N_{[\text{dis}_B(S), B]}^+(U)| + Ct|E(U)| + Ct \sum_{Y \in \mathcal{T}(X)} |Q_Y|,$$

Moreover, $\sum_{Y \in \mathcal{T}(X)} |Q_Y| \leq |U|(\delta + l/t)$.

Thus in the top layer when called with $B = \infty, S = \{s\}$ and $l = \lceil (\log n)/t \rceil$, it takes time $O(m(t + \frac{\log n \log t}{\delta t}))$, optimized to $O(m\sqrt{\log n} + \sqrt{mn \log n \log \log n})$ with best parameters.

Proof. We first prove the time of one BMSSP call by induction. Clearly base case satisfies Lemma 3.9.

Now assume $l > 0$. By induction, sub-calls satisfy Lemma 3.9 and take time

$$T_0 = C \sum_i \left(|U_i|(l-1 + \log t) \log t + t|N_{[\text{dis}_{B_i}(S_i), B_i]}^+(U_i)| + t|E(U_i)| \right) + Ct \sum_{Y \in \mathcal{T}^\circ(X)} |Q_Y|.$$

By Lemma 3.8, $\sum_i |U_i| \leq |U|$. By Lemma 3.7, in the i -th iteration, $\langle \emptyset, S_i \rangle$ is a frontier for $\tilde{U}(B_i, S_i)$ and S_i is chosen from $\mathcal{D} \cup P$, so $\text{dis}_{B_i}(S_i) = d_{B_i}[S_i] \geq d_B[\mathcal{D} \cup P] \geq B'_{i-1}$. Thus

$$T_0 \leq C|U|(l-1 + \log t) \log t + Ct \sum_i |N_{[B'_{i-1}, B_i]}^+(U_i)| + Ct \sum_i |E(U_i)| + Ct \sum_{Y \in \mathcal{T}^\circ(X)} |Q_Y|.$$

In a full execution, $U = \tilde{U}$. In a partial execution, $|U| > t|S|$. In both cases, $|U| \geq \min\{t|S|, |\tilde{U}|\}$.

We pick $C' = C/100$ to bound the time constant directly consumed by X and list them sequentially:

1. By Remark 3.3, $p \leq \min\{|S|, |\tilde{U}|/k\} \leq |U|/k$. Thus FindPivots (Line 4) and initial insertion (Line 7) take time $T_1 = C'|U| \log t + C't|Q|$.
2. Picking $\{x \in P_j : d[x] < B_i\}$ (Line 13) takes $C'k$ time for each i and j such that $p_j \in S_i$.
 - If i -th sub-call is a partial execution, $T_{2,par,i} \leq C'k|S_i| \leq C'|U_i|$.
 - If i -th sub-call is a full execution, $p_j \in U_i$ is removed from P_j (Line 20), causes re-selection and insertion of new p_j (Line 29) of $O(t)$ time and can be absorbed by it (Item 5); except if P_j becomes empty, but this happens at most one time for each j : $T_{2,full} \leq C'pk = C'|U|$.

In total $T_2 \leq C' \sum_i |U_i| + C'|U| = 2C'|U|$.

3. Iterating U_i , updating J , PULL and MERGE operations and checking edges of U_i and W' (Line 20,10,15,23, 36) together take linear time $T_3 = C'\delta|U| \leq C'|U| \log t$.
4. Insertion of v from edges (u, v) in $N_{[B_i, B]}^+(U_i)$ (Line 24) takes time $T_4 = C't \sum_i |N_{[B_i, B]}^+(U_i)|$.
5. Re-selection and insertion of each new p_j (Line 29) take time $O(k + t) \leq C't$.

- If X is a partial execution, $T_{5,par} = C't|S| \leq C'|U|$.
- Below we assume X is a full execution. For each P_j , in the i -th iteration, if it spends $C't$ time (Line 29) to re-select and insert a new p_j , its old p'_j must have been in U_i and removed (Line 20). For each j , P_j has at most one such p'_j removed from each U_i . Denote g_j the number of such p'_j 's, and then the time is $T_{5,full} = C't \sum_{j=1}^p g_j$. By Lemma 3.8, U_i 's are disjoint. For each j , those p'_j 's over different iterations distribute in g_j distinct U_i 's.

By Lemma 3.2, each P_j is connected by an undirected tree F_j whose vertices are in $\tilde{U} = U$. Denote $\mathcal{E} = E(U) \setminus (\bigcup_i E(U_i))$, the set of edges connecting vertices from two different U_i 's. Removing edges of $\mathcal{E}$ from F_j splits it into at least g_j weakly connected components. Thus F_j contains at least $g_j - 1$ edges in $\mathcal{E}$. As F_j 's are edge-disjoint and $tp \leq |U|t/k = |U| \log t$,

$$T_{5,full} = C't \sum_{j=1}^p g_j \leq C'tp + C't \sum_{j=1}^p |F_j \cap \mathcal{E}| \leq C'|U| \log t + C't|\mathcal{E}|.$$

6. Insertion of vertices in $\{x \in S : d[x] \in [B', B)\}$ (Line 33) takes time $O(t|S|)$, but it is non-zero only if this call is a partial execution ($B' < B$) such that $|S| \leq |U|/t$, so $T_6 = C't|S| \leq C'|U|$.

7. Insertion of v from edges (u, v) in $N_{[B', B]}^+(W')$ (Line 37) takes time $T_7 = C't|N_{[B', B]}^+(W')|$.

To sum up, because $B'_{i-1} \geq B'_0 \geq d_B[S] = \text{dis}_B(S)$, U is a disjoint union of U_i 's and W' , and $|\mathcal{E}| + \sum_i |E(U_i)| \leq |E(U)|$, the time of one BMSSP call is

$$\begin{aligned}
T &\leq T_0 + T_1 + T_2 + T_3 + T_4 + T_{5,par} + T_{5,full} + T_6 + T_7 \\
&\leq (C|U|(l-1+\log t)\log t + 7C'|U|\log t) \\
&\quad + \left(C't|N_{[B', B]}^+(W')| + \sum_i \left(Ct|N_{[B'_{i-1}, B_i]}^+(U_i)| + C't|N_{[B_i, B]}^+(U_i)| \right) \right) \\
&\quad + \left(C't|\mathcal{E}| + Ct \sum_i |E(U_i)| \right) + \left(Ct \sum_{Y \in \mathcal{T}^o(X)} |Q_Y| + C't|Q_X| \right) \\
&\leq C|U|(l+\log t)\log t + Ct|N_{[\text{dis}_B(S), B]}^+(U)| + Ct|E(U)| + Ct \sum_{Y \in \mathcal{T}(X)} |Q_Y|.
\end{aligned}$$

Now we show that, $\sum_{Y \in \mathcal{T}(X)} |Q_Y| \leq |U|(\delta + l/t)$.

- For a partial execution Y , $|Q_Y| \leq |U_Y|/t$. The U_Y 's for Y under $\mathcal{T}(X)$ on each layer are disjoint and their union is a subset of U . Summing over all layers we obtain $\sum_{\text{partial } Y \in \mathcal{T}(X)} |Q_Y| \leq |U|l/t$.
- Below we assume Y is a full execution. For any $v \in Q_Y$, think of the latest valid relaxation (Line 24 or Line 37 in Algorithm 3, or Line 8 in Algorithm 4) on edge (u, v) for some complete vertex u which caused insertion of v . Then u is already complete before the execution of Y .

If $v \in Q_Z$ for another full execution Z , since $v \in U_Y$ and also $v \in U_Z$, $Z \in \mathcal{T}(Y)$ or $Y \in \mathcal{T}(Z)$. Without loss of generality, assume $Z \in \mathcal{T}(Y)$. Because $v \in Q_Y$ is not passed down to descendants of Y in $\mathcal{T}(Y)$, and also u is not in U_Y , so $v \in Q_Z$ must be owing to another edge (u', v) relaxed in some descendant of Y . Therefore, the number of appearances of v in Q_Y of full executions Y is bounded by the number of relaxations from its complete predecessors, which is bounded by δ .

Thus $\sum_{\text{full } Y \in \mathcal{T}(X)} |Q_Y| \leq \delta|U|$.

To sum up, note that both $|N^+(U)|$ and $|E(U)|$ are bounded by $O(\delta|U|)$, a call X of BMSSP runs in time $O(|U|(l\log t + \delta t))$. Recall we work on a graph with $O(m)$ edges, $O(m/\delta)$ vertices and max degree δ , with $l = \lceil (\log n)/t \rceil$, so in the top layer the total time is $O(m(t + \frac{\log n \log t}{\delta t}))$.

The best³ parameter is $t = \lceil \sqrt{\log n \log \log n / \delta} \rceil$ and $\delta = \frac{1}{4} \min\{\frac{m}{n}, \log \log n\}$. If $\frac{m}{n} \leq \log \log n$, the total time is $O(\sqrt{mn \log n \log \log n})$. If $\frac{m}{n} > \log \log n$, the total time is $O(m\sqrt{\log n})$. $\square$

3.7 Base Case

The base case is solved using Dijkstra's algorithm. It is efficient enough to spend $\log t$ time on each edge.

³“best” in terms of order analysis, the specific constant, though not a concern of this paper, may be adjusted in implementations.

Algorithm 4: Base Case of BMSSP (Algorithm 3) when $l = 0$

```
1 Initialize normal binary search tree  $\mathcal{D} \leftarrow \{\langle x, d[x] \rangle : x \in S\}$  (parameterized by  $M = 1$ )
2  $U \leftarrow \emptyset$ 
3 while  $\mathcal{D}.\text{NON-EMPTY}()$  and  $|U| \leq t^3$  do //  $|\mathcal{D}| \leq t^3\delta \leq t^4$ , each operation  $O(\log t)$  time.
4    $u, B' \leftarrow \mathcal{D}.\text{PULL}()$ 
5    $U \leftarrow U \cup \{u\}$ 
6   forall edge  $(u, v)$  do //  $u$  is complete
7     if  $\text{Relax}(u, v, B)$  then
8        $\mathcal{D}.\text{INSERT}(v, d[v])$  // On duplicate, keep the smaller one.
9 return  $B', U, \mathcal{D}$ 
```

A Missing Proofs

A.1 Partition

In this section we introduce the tree partition algorithm used for pivot selection. We slightly modify the topological partition algorithm in [Fre83], which is also used in [DLWX18]. Note that in the algorithm T is a directed tree but we treat every edge in T as undirected.

Algorithm 5: Tree partition

```
1 Function Partition(tree  $T$ ):
2    $r \leftarrow \text{root of } T$ 
3    $U \leftarrow \{r\}$ 
4   forall subtree  $T'$  do
5      $U \leftarrow U \cup \text{Partition}(T')$ 
6     if  $|U| \geq s$  then
7       Report a new group  $U$ 
8        $U \leftarrow \{r\}$ 
9   return  $U$ 
```

Lemma A.1 (Partition). *Given a tree T with n vertices and an integer $s \in [1, n]$, we can partition T into edge-disjoint subtrees $T_1, T_2, \dots, T_p$ in linear time, such that $|T_j| \in [s, 3s)$ for all j .*

Proof. Algorithm 5 outlines the tree partition procedure. We select an arbitrary vertex as the root. The main algorithm calls $\text{Partition}(T)$ that performs a depth-first search on T , and collects all the groups reported as $T_1, T_2, \dots, T_p$ in order. The U returned by $\text{Partition}(T)$ is merged into T_p , or regarded as a new group if no group is reported before.

$\text{Partition}(T)$ always returns a subset of vertices in T , including the root r . Prior to the merge, by induction we can deduce that each T_j and the final U induce a subtree, and each edge in T appears in exactly one of them. If the merge occurs, let r_p be the root node when T_p is reported, then r_p must be contained in both T_p and U , thus T_p after the merge can still induce a subtree in T .

It's clear that the set U returned in line 9 has size $[1, s]$, therefore any group reported has size $[s, 2s)$. After the final merge the size of T_p remains less than $3s$. $\square$

A.2 Proof of the Data Structure

Lemma A.2 (Lemma 3.4 restated). *Given at most N key/value pairs involved, a parameter M , and an upper bound B , there exists a data structure $\mathcal{D}$ that supports the following operations:*

Insert *Insert a key/value pair. If the key already exists, keep the one with smaller value.*

Merge *For another data structure $\mathcal{D}'$ with smaller parameter $M'(< M/3)$ as specified in this lemma, such that all the values in $\mathcal{D}'$ are smaller than all the values in $\mathcal{D}$, insert all pairs of $\mathcal{D}'$ into $\mathcal{D}$ (keep the smaller value for duplicate keys).*

Pull *Return a subset S' of keys where $|S'| \leq M$ associated with the smallest $|S'|$ values and an upper bound x that separates S' from the remaining values in $\mathcal{D}$. Specifically, if there are no remaining values, $x = B$; otherwise, $|S'| = M$.*

If $M = 1$, INSERT and PULL take $O(\log N)$ time with MERGE unsupported, for base case only (Section 3.7).

If $M > 1$, we require $M \geq \log(N/M)$. INSERT takes $O(\log(N/M))$ amortized time; other operations are linear: MERGE takes $O(|\mathcal{D}'|)$ time and PULL takes $O(|S'|)$ time.

Proof. We use a self-balanced binary search tree (e.g. Red-Black Tree [GS78]) of blocks of size $\Theta(M)$.

If $M = 1$ in base case, a normal BST suffices, and below we assume $M \leq \log(N/M)$.

We require blocks holding disjoint intervals that values stored in each block suit its interval, and blocks are ordered by their intervals. Inside each block, key/value pairs are maintained by unordered linked lists.

We store a table from key to the block linked list node that corresponds to it, to support $O(1)$ membership inspection and deletion; keep track of the first BST block⁴ for $O(1)$ access and the total number of key/value pairs present in $\mathcal{D}$ used below. In the beginning, $\mathcal{D}$ is initialized with a single empty block of $[0, B)$.

Insert Check if v already exists, and continue only if the new value is smaller. Insert the pair to the new block and delete its old occurrence (if any). This takes $O(\log(N/M))$ time.

Merge If $\mathcal{D}'$ is empty, do nothing. If $0 < |\mathcal{D}'| < M$, restructure $\mathcal{D}'$ to one block and append to the first block of $\mathcal{D}$, possibly inducing block normalization introduced below. Now we assume $|\mathcal{D}'| \geq M$.

Scan blocks of $\mathcal{D}'$ in ascending order, keep the smaller value for duplicates; whenever we collect at least $M/3$ elements or $\mathcal{D}'$ is exhausted, insert them into $\mathcal{D}$. Causing $O(|\mathcal{D}'|/M)$ insertions of $O(\log(N/M)) = O(M)$ time each, in total we spend $O(|\mathcal{D}'|)$ time.

Pull If $\mathcal{D}$ contains no more than M elements, pulling all of them takes linear time. Otherwise, continuously extract the smallest block from $\mathcal{D}$ until we have at least $M + 1$ but at most $2M$ elements. Find the $(M + 1)$ -th element x in linear time using median-find algorithm ([BFP⁺73]). Output the smaller M elements and x . Join remaining elements larger than x to the current smallest block. If the size of that block exceeds M , split it into two blocks and insert them back to $\mathcal{D}$. All operations above take $O(\log(N/M) + M) = O(M)$ as $\log(N/M) \leq M$.

Normalization We normalize block sizes for better performance guarantees after any operation above, resulting in $O(1)$ extra time per operated key/value pair. After INSERT and MERGE operation, we keep the operated block size in interval $[M/3, M]$; after PULL and normalization, in interval $[M/2, 2M/3]$; or if in total there are less than $M/3$ elements, restructure $\mathcal{D}$ to contain only one block.

Each inserted and merged pair results in at most one pair membership change in at most two blocks. When the constraint is violated, the block has interacted with $\Theta(M)$ newly inserted or merged pairs

⁴This is used when $|\mathcal{D}'|$ is too small described in the merge section.

since last pull or normalization. We join it to an adjacent block if it is under-sized, or split the block into two equally sized blocks using median-find algorithm if it is over-sized. In total each normalization takes $O(M)$ time and can be amortized to $O(1)$ time per inserted or merged pair.

□

References

- [BCF23] Karl Bringmann, Alejandro Cassis, and Nick Fischer. Negative-Weight Single-Source Shortest Paths in Near-Linear Time: Now Faster! . In *2023 IEEE 64th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 515–538, Los Alamitos, CA, USA, November 2023. IEEE Computer Society.
- [Bel58] Richard Bellman. On a routing problem. *Quarterly of Applied Mathematics*, 16:87–90, 1958.
- [BFP⁺73] Manuel Blum, Robert W. Floyd, Vaughan Pratt, Ronald L. Rivest, and Robert E. Tarjan. Time bounds for selection. *Journal of Computer and System Sciences*, 7(4):448–461, 1973.
- [BNWN22] Aaron Bernstein, Danupon Nanongkai, and Christian Wulff-Nilsen. Negative-weight single-source shortest paths in near-linear time. In *2022 IEEE 63rd Annual Symposium on Foundations of Computer Science (FOCS)*, pages 600–611, 2022.
- [CKL⁺22] Li Chen, Rasmus Kyng, Yang P. Liu, Richard Peng, Maximilian Probst Gutenberg, and Sushant Sachdeva. Maximum flow and minimum-cost flow in almost-linear time. In *2022 IEEE 63rd Annual Symposium on Foundations of Computer Science (FOCS)*, pages 612–623, 2022.
- [DI04] Camil Demetrescu and Giuseppe F. Italiano. A new approach to dynamic all pairs shortest paths. *J. ACM*, 51(6):968–992, November 2004.
- [Dij59] E. W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1:269–271, 1959.
- [DLWX18] Ran Duan, Kaifeng Lyu, Hongxun Wu, and Yuanhang Xie. Single-source bottleneck path algorithm faster than sorting for sparse graphs. *CoRR*, abs/1808.10658, 2018.
- [DMM⁺25] Ran Duan, Jiayi Mao, Xiao Mao, Xinkai Shu, and Longhui Yin. Breaking the sorting barrier for directed single-source shortest paths. In *Proceedings of the 57th Annual ACM Symposium on Theory of Computing*, STOC ’25, page 36–44, New York, NY, USA, 2025. Association for Computing Machinery.
- [DMSY23] R. Duan, J. Mao, X. Shu, and L. Yin. A randomized algorithm for single-source shortest path on undirected real-weighted graphs. In *2023 IEEE 64th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 484–492, Los Alamitos, CA, USA, November 2023. IEEE Computer Society.
- [Fin24] Jeremy T. Fineman. Single-source shortest paths with negative real weights in $\tilde{O}(mn^{8/9})$ time. In *Proceedings of the 56th Annual ACM Symposium on Theory of Computing*, page 3–14, Chicago, IL, USA, 2024.
- [Fre83] Greg N. Frederickson. Data structures for on-line updating of minimum spanning trees. In *Proceedings of the Fifteenth Annual ACM Symposium on Theory of Computing*, STOC ’83, page 252–257, New York, NY, USA, 1983. Association for Computing Machinery.

- [FT87] M. L. Fredman and R. E. Tarjan. Fibonacci heaps and their uses in improved network optimization algorithms. *JACM*, 34(3):596–615, 1987.
- [FW93] Michael L. Fredman and Dan E. Willard. Surpassing the information theoretic bound with fusion trees. *Journal of Computer and System Sciences*, 47(3):424–436, 1993.
- [FW94] Michael L. Fredman and Dan E. Willard. Trans-dichotomous algorithms for minimum spanning trees and shortest paths. *Journal of Computer and System Sciences*, 48(3):533–551, 1994.
- [GS78] Leo J. Guibas and Robert Sedgwick. A dichromatic framework for balanced trees. In *19th Annual Symposium on Foundations of Computer Science (sfcs 1978)*, pages 8–21, 1978.
- [Hag00] Torben Hagerup. Improved shortest paths on the word RAM. In Ugo Montanari, José D. P. Rolim, and Emo Welzl, editors, *Automata, Languages and Programming*, pages 61–72, Berlin, Heidelberg, 2000. Springer Berlin Heidelberg.
- [HHR⁺24] Bernhard Haeupler, Richard Hladík, Václav Rozhoň, Robert E. Tarjan, and Jakub Tětek. Universal optimality of dijkstra via beyond-worst-case heaps. In *2024 IEEE 65th Annual Symposium on Foundations of Computer Science (FOCS)*, 2024.
- [HJQ25a] Yufan Huang, Peter Jin, and Kent Quanrud. Faster negative length shortest paths by bootstrapping hop reducers. *CoRR*, abs/2506.00428, 2025.
- [HJQ25b] Yufan Huang, Peter Jin, and Kent Quanrud. Faster single-source shortest paths with negative real weights via proper hop distance. In *Proceedings of the 2025 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 5239–5244, Chicago, IL, USA, 2025.
- [PR05] Seth Pettie and Vijaya Ramachandran. A shortest path algorithm for real-weighted undirected graphs. *SIAM Journal on Computing*, 34(6):1398–1431, 2005.
- [Ram96] Rajeev Raman. Priority queues: Small, monotone and trans-dichotomous. In *Proceedings of the Fourth Annual European Symposium on Algorithms, ESA '96*, page 121–137, Berlin, Heidelberg, 1996. Springer-Verlag.
- [Ram97] Rajeev Raman. Recent results on the single-source shortest paths problem. *SIGACT News*, 28(2):81–87, June 1997.
- [Tho99] Mikkel Thorup. Undirected single-source shortest paths with positive integer weights in linear time. *J. ACM*, 46(3):362–394, May 1999.
- [Tho00a] Mikkel Thorup. Floats, integers, and single source shortest paths. *J. Algorithms*, 35(2):189–201, May 2000.
- [Tho00b] Mikkel Thorup. On RAM priority queues. *SIAM Journal on Computing*, 30(1):86–109, 2000.
- [Tho04] Mikkel Thorup. Integer priority queues with decrease key in constant time and the single source shortest paths problem. *Journal of Computer and System Sciences*, 69(3):330–353, 2004. Special Issue on STOC 2003.
- [Yan26] Shuyi Yan. Lossless derandomization for undirected single-source shortest paths and approximate distance oracles. In *SIAM Symposium on Simplicity in Algorithms (SOSA 2026)*, Vancouver, Canada, January 12–14, 2026. SIAM, 2026.